

Homework 2

Youssef Amin

October 2025

Problem 1: Implementing a Stack with a BST

The goal of this problem is to implement a stack using a Binary Search Tree (BST). Each node is ordered by its `value`, while a class variable `order` tracks the insertion sequence to emulate stack (LIFO) behavior. When popping, the tree searches for the node with the highest order number, removes it, and decrements `order`. Inserting by value will preserve $O(\log n)$ time when searching for the top node. If you order it by the order number you just have a linked list and that's $O(n)$ which is no good. Really I don't believe anyone would ever implement a stack with a tree in the first place.

```
1 class Node:
2     def __init__(self, val, index):
3         self.val = val
4         self.index = index
5         self.right = None
6         self.left = None
7
8 class StackBST:
9     order = 0
10
11     def __init__(self):
12         self.root = None
13
14     def push(self, x):
15         """
16         Assigns numerical order to nodes and appropriately places them in the tree by
17         ↪ value.
18         """
19         StackBST.order += 1
20         node = Node(x, StackBST.order)
21
22         if self.root is None:
23             self.root = node
24         return
```

```
25     curr = self.root
26     while True:
27         if node.val > curr.val:
28             if curr.right:
29                 curr = curr.right
30             else:
31                 curr.right = node
32                 break
33         else:
34             if curr.left:
35                 curr = curr.left
36             else:
37                 curr.left = node
38                 break
39
40     def pop(self):
41         """
42         Searches for the largest order number in the tree,
43         removes it, and returns its value.
44         """
45         if self.root is None:
46             return None
47
48         target = StackBST.order
49
50         def find_node(node, parent=None):
51             if node is None:
52                 return None
53
54             if node.index == target:
55                 if parent is None:
56                     self.root = node.left or node.right
57                 else:
58                     if parent.left == node:
59                         parent.left = node.left or node.right
60                     else:
61                         parent.right = node.left or node.right
62                 return node
63
64             found = find_node(node.left, node)
65             if found:
66                 return found
67             return find_node(node.right, node)
68
69         found_node = find_node(self.root)
70         if found_node is None:
71             return None
72
73         StackBST.order -= 1
```

```
74     return found_node.val
```

Test

```
1  stack_bts = StackBST()
2  stack_bts.push(15)
3  stack_bts.push(2)
4  stack_bts.push(30)
5
6  nums = [stack_bts.pop(), stack_bts.pop(), stack_bts.pop()]
7  print(nums)
```

Output: [30, 2, 15]

Problem 2: Combining Heaps

For combining heaps, I wrote a function called `heapify()` which, starting from the last parent node, iterates toward 0. If the current value at `i` is greater than its children it swaps with the smaller of the two.

```
1  def heapify(heap):
2      """
3      Starting from the last parent node
4      iterate upwards swapping elements with the smallest of its two children
5      """
6      last_parent = len(heap)//2 - 1
7
8      for i in range(last_parent, -1, -1):
9          while True:
10             l = 2 * i + 1
11             r = 2 * i + 2
12             smallest = i
13
14             if l < len(heap) and heap[l] < heap[smallest]:
15                 smallest = l
16             if r < len(heap) and heap[r] < heap[smallest]:
17                 smallest = r
18
19             if smallest == i:
20                 break
21             heap[i], heap[smallest] = heap[smallest], heap[i]
22             i = smallest
```

Since there are way more nodes near the bottom of the tree, the number of swaps required for each node decreases exponentially. Although each parent can take up to $O(\log n)$ time to sift down, the vast majority of nodes only move one or two levels. Summing over all nodes gives a total cost of $O(n)$.

Test

```

1 heap1 = [5, 4, 3, 8, 6]
2 heap2 = [3, 7, 4, 9, 5, 7]
3 merged = heap1 + heap2
4 heapify(heap1)
5 heapify(heap2)
6 heapify(merged)
7 print(f"heap1: {heap1}")
8 print(f"heap2: {heap2}")
9 print(f"merged heap: {merged}")

```

Output:

```

heap1:  [3, 4, 5, 8, 6]
heap2:  [3, 5, 4, 9, 7, 7]
merged heap:  [3, 4, 3, 4, 5, 5, 7, 8, 9, 6, 7]

```

Problem 3: Merging Binary Search Trees

First, I implemented a binary search tree class. After constructing two separate trees, my algorithm performs an in order traversal on each, producing two sorted lists in $O(m + n)$ time. I then merge these two lists by iterating through them and repeatedly adding the smaller of the two current elements to a new list that represents the in order traversal of the merged tree. This merging step also takes $O(m + n)$ time. Combining both operations results in an overall time complexity of $O(m + n)$. The key functions used in this process are `merge_trees()` and `in_order_traversal()`.

```

1 class BST:
2     def __init__(self):
3         self.root = None
4
5     def insert(self, x: int):
6         """
7         Creates a new node from a given integer and inserts it into the binary tree
8         by checking the value of the index and inserting it in the first available
9         and appropriate spot.
10        It finds the correct spot by recursive descent.
11        """

```

```
12     new_node = Node(x, None)
13
14     if self.root is None:
15         self.root = new_node
16         return
17
18     def find_n_place(node: Node):
19         val = node.val
20         if val > x:
21             if node.left is None:
22                 node.left = new_node
23             else:
24                 find_n_place(node.left)
25         elif val < x:
26             if node.right is None:
27                 node.right = new_node
28             else:
29                 find_n_place(node.right)
30
31     find_n_place(self.root)
32
33     def in_order_traversal(self):
34         """
35         Recursively descend left to the smallest node.
36         Add node to list and then check right branch of each node.
37         """
38         ordered = []
39
40         def traverse(node: Node):
41             if node:
42                 traverse(node.left)
43                 ordered.append(node.val)
44                 traverse(node.right)
45
46         traverse(self.root)
47         return ordered
```

```
1 def make_tree_from_list(lst: list) -> BST:
2     """
3     This just makes it easier for me to create binary trees
4     and isn't involved in my algorithm.
5     """
6     bst = BST()
7     for i in lst:
8         bst.insert(i)
9     return bst
```

```
1 def merge_trees(o1, o2):
2     """
3     This method takes two lists which represent in-order traversals of trees.
4     It loops through the trees adding the smaller of the two values to a new list
5     which represents an in-order traversal of a new merged binary tree.
6     """
7     merged = []
8     j = 0
9     i = 0
10    while i < len(o1) and j < len(o2):
11        if o1[i] < o2[j]:
12            merged.append(o1[i])
13            i += 1
14        else:
15            merged.append(o2[j])
16            j += 1
17
18    merged.extend(o1[i:])
19    merged.extend(o2[j:])
20
21    return merged
```

Test

```
1 order1 = tree1.in_order_traversal()
2 order2 = tree2.in_order_traversal()
3 merged = merge_trees(order1, order2)
4
5 print(f"in order tree1: {order1}")
6 print(f"in order tree2: {order2}")
7 print(f"tree1 and tree2 merged: {merged}")
```

Output:

in order tree1: [1, 3, 5, 7, 9]

in order tree2: [2, 4, 6, 8]

tree1 and tree2 merged: [1, 2, 3, 4, 5, 6, 7, 8, 9]

Problem 4: Partitioning Selection Algorithm

After investigating partitioning into groups of six, it is safe to say the algorithm remains linear. I chose the third element in each group as the median, resulting in the recurrence

$$T(n) = T\left(\frac{n}{6}\right) + T\left(\frac{3n}{4}\right) + O(n).$$

We partition into groups of six, giving us approximately $n/6$ medians. Sorting each group requires $O(n)$ total time.

Half of the medians are less than or equal to the median of medians, so there are $n/12$ such groups. Each of these contributes at least three elements smaller than or equal to the pivot, giving $3(n/12) = n/4$ elements guaranteed less than the pivot. Similarly, for the upper half, each group contributes up to four elements greater than or equal to the pivot, giving $4(n/12) = n/3$ elements greater than the pivot. In the worst case, we recurse on the larger side of size $n - n/4 = 3n/4$.

Using the recursion tree method, we split into subproblems of sizes $n/6$ and $3n/4$, which sum to $(11/12)n$. Expanding one more level gives

$$\frac{n}{36} + \frac{n}{8} + \frac{n}{8} + \frac{9n}{16} = \left(\frac{11}{12}\right)^2 n.$$

Thus, the ratio per level is $r = 11/12$, forming a geometric series:

$$\sum_{i=0}^h r^i = \left(\frac{1 - r^{n+1}}{1 - r}\right) \rightarrow n \rightarrow \infty \frac{1}{1 - r} = 12n = O(n), r = \frac{11}{12}.$$

Therefore, the selection algorithm with groups of six still runs in $\Theta(n)$ linear time.

Problem 5: Longest Increasing Subsequence

For this problem I created two lists. the first keeps track of the length of the largest subsequence, and the second keeps track of the previous index of any larger number. I loop through the sequence with $i = 1$ to `length` and $j = 0$ to i . If the sequence at i is greater than the sequence at j , I check that `largest[i]` is less than `largest[j] + 1`; if it is, I replace `largest[i]` with `largest[j] + 1`. I also update the previous index. After that, I traverse the list starting from the index of the largest subsequence and trace backward to reconstruct the increasing sequence.

```
1 def find_mono_increasing(sequence):
2     """
3     Using two lists, track the longest increasing subsequence
4     Loop through sequence for each index i starts at 1 and j starts at 0
5     check if the sequence at i is greater than the sequence at j
6     if it is add the lessers largest counter to the current largest counter
7     record the index of the element that the current element is larger than at the
8     → current
9     elements previous
10    This creates a back path that we can use to construct the largest monoincreasing
11    → subsequence
12    """
13    length = len(sequence)
14    largest = [1] * length
15    previous = [-1] * length
16
17    for i in range(1, length):
18        for j in range(0, i):
19            if sequence[i] > sequence[j]:
20                if largest[i] < largest[j] + 1:
21                    largest[i] = largest[j] + 1
22                    previous[i] = j
23
24    max_i = largest.index(max(largest))
25    subsequence = []
26    while max_i >= 0:
27        subsequence = [sequence[max_i]] + subsequence
28        max_i = previous[max_i]
29
30    return subsequence
```

Test

```
1 A = [3, 10, 2, 1, 20]
2 B = find_mono_increasing(A)
3 print(f"A: {A}")
4 print(f"B: {B}")
```

Output:

A: [3, 10, 2, 1, 20] B: [3, 10, 20]

Problem 6: Sort By Permutation

My implementation runs in $O(n)$ time by looping through both given sequences A and w , checking whether the permutation sequence w is in order. If it is not, I swap the elements of A according to the indices specified by w . This process only requires a single pass through the arrays, performing swaps along the way, which results in linear time complexity and constant extra space.

```
1 def order_by_permutation(A, w):
2     """
3     Reorder A in place according to permutation w
4     """
5     n = len(A)
6     i = 0
7     while i < n:
8         if w[i] != i + 1:
9             j = w[i] - 1
10            A[i], A[j] = A[j], A[i]
11            w[i], w[j] = w[j], w[i]
12        else:
13            i += 1
```

Test

```
1 A = [10.5, 9.3, 2.7, 13.6]
2 w = [4, 2, 3, 1]
3 order_by_permutation(A, w)
4 print(f"A: {A}")
5 print(f"w: {w}")
6
7 B = [2, 4, 6, 8]
8 mu = [4, 3, 2, 1]
9 order_by_permutation(B, mu)
10 print(f"B: {B}")
11 print(f"mu: {mu}")
```

Output:

```
A: [13.6, 9.3, 2.7, 10.5]
w:  [1, 2, 3, 4]
B: [8, 6, 4, 2]
mu: [1, 2, 3, 4]
```

Problem 7: Double Knapsack Problem

My idea is to extend the dynamic programming approach used in the 0/1 Knapsack problem by creating a three dimensional table `table[i][w1][w2]`, where `i` represents the i -th item, `w1` is the capacity of the first knapsack, and `w2` is the capacity of the second knapsack.

At each iteration, the algorithm considers three possibilities for item i : skipping it, placing it in the first knapsack, or placing it in the second knapsack. This three dimensional table effectively tracks all possible distributions of capacity between the two knapsacks.

Let w_i denote the weight of item i , and p_i denote its value. The recurrence relation is:

$$V[i, w_1, w_2] = \max(V[i-1, w_1, w_2], V[i-1, w_1 - w_i, w_2] + p_i, V[i-1, w_1, w_2 - w_i] + p_i)$$

The algorithm's overall time complexity is $O(nW^2)$, just like the standard dynamic programming approach for the single knapsack problem.

Credit to Abdule Bari's explanation of the 0/1 knapsack dynamic programming solution:
<https://www.youtube.com/watch?v=nLmhmb6NzcM&t=1215s>

Problem 8: Finding Contained Intervals

For this problem I sort the given list of intervals by the smallest starting point and the largest ending point which I achieved by setting the end point to negative for the sort. Sorting takes $O(n \log n)$ time. Next I make an endpoint variable set to $-\infty$ for each range in the given list I check if the end point is larger than the current endpoint. if it is I set the endpoint equal to this new largest endpoint. If its not larger I add the range to a list. This finds all the intervals that are contained in other intervals since each intervals starting point will be greater than its previous we know the interval is contained if its endpoint is less than the previous intervals largest endpoint. Looping through all the intervals takes $O(n)$ time therefor we achieve $O(n \log n)$.

```

1 def find_contained_intervals(intervals):
2     intervals.sort(key=lambda x: (x[0], -x[1]))
3
4     contained = []
5     end_point = float('-inf')
6
7     for x1, x2 in intervals:
8         if x2 <= end_point:
9             contained.append((x1,x2))
10        else:
11            end_point = x2
12    return contained

```

Test

```
1 intervals1 = [(1, 2), (3, 6), (4, 5), (0, 7)]
2 print(f" Original Intervals: {intervals1}")
3 contained1 = find_contained_intervals(intervals1)
4 print(f" Contained Intervals: {contained1}\n")
5
6 intervals2 = [(0, 2), (3, 6), (4, 5), (3, 7)]
7 print(f" Original Intervals: {intervals2}")
8 contained2 = find_contained_intervals(intervals2)
9 print(f" Contained Intervals: {contained2}\n")
10
11 intervals3 = [(0, 1), (3, 6), (4, 11), (3, 7)]
12 print(f" Original Intervals: {intervals3}")
13 contained3 = find_contained_intervals(intervals3)
14 print(f" Contained Intervals: {contained3}")
```

Output:

```
Original Intervals: [(1, 2), (3, 6), (4, 5), (0, 7)]
Contained Intervals: [(1, 2), (3, 6), (4, 5)]
Original Intervals: [(0, 2), (3, 6), (4, 5), (3, 7)]
Contained Intervals: [(3, 6), (4, 5)]
Original Intervals: [(0, 1), (3, 6), (4, 11), (3, 7)]
Contained Intervals: [(3, 6)]
```